COUNT MASTER

Autonomous Build Specification for a Unity MCP Agent

3D Crowd-Runner · Unity 2022 LTS · Built-in Render Pipeline · Android

Purpose of this document. This is a complete, self-contained build order. An AI agent driving Unity through the MCP tool should be able to construct the entire game from this file alone: creating the folder structure, all C# scripts (provided verbatim below), the prefabs, the three level scenes, and the Inspector wiring. Every script is drop-in ready. Every wiring step is explicit. Follow the phases in order.

0. Instructions for the Building Agent

Read this section first. It defines how you, the Unity MCP agent, should execute this document.

Hard constraints (never violate)

- **Engine:** Unity 2022 LTS. **Render pipeline:** Built-in only — do NOT install or convert to URP/HDRP.
- **Platform:** Android. Set Build Settings platform to Android before finishing.
- **Architecture:** Basic OOP with plain MonoBehaviours. Do NOT introduce interfaces, abstract classes, dependency injection, ScriptableObject architectures, or any SOLID-driven layering. Keep it junior-readable.
- **Managers:** GameManager, LevelManager, UIManager, CrowdManager, ObjectPool each use the singleton pattern (one static `Instance`).
- **Pooling:** Only PlayerCharacter is pooled, and the pool is pre-warmed at scene start.
- **Scripts:** Use the code in Section 6 verbatim. Do not rewrite or 'improve' it. Every method already has a comment.
- **Designer values:** Gate operator + value are set per prefab instance in the Inspector — never randomised at runtime.

Execution order (phases)

- **Phase 1** — Create the folder structure (Section 4).
- **Phase 2** — Create all 15 C# scripts exactly as given (Section 6). Let them compile clean before continuing.
- **Phase 3** — Set up TextMeshPro (Window > TextMeshPro > Import TMP Essentials) and create the required tags/layers (Section 5).
- **Phase 4** — Build the prefabs and attach scripts (Section 7).
- **Phase 5** — Assemble Level 1, then Level 2, then Level 3 scenes (Section 8).
- **Phase 6** — Wire all Inspector references (Section 9), then run the acceptance checklist (Section 10).

If something is ambiguous

Prefer the simplest implementation that satisfies the described behaviour. Placeholder primitive art (capsules for characters, cylinders for hurdles, cubes for gates) is acceptable — gameplay correctness and clean wiring matter more than final art. Expose every tunable number as a serialized field so it can be adjusted later.

1. Tech Stack & Constraints

Engine	Unity 2022 LTS
Render Pipeline	Built-in Render Pipeline (no URP / HDRP)
Platform	Android
Pattern	Singleton for all managers
Pooling	Object pooling for crowd characters (PlayerCharacter only)
Code Style	Basic OOP — NO SOLID, NO interfaces, NO abstraction layers
Text UI	TextMeshPro (count bubble + panels)
Developer Level	Junior-friendly — clear, commented MonoBehaviours

2. Game Overview

Count Master is a 3D crowd-runner for Android. The player controls a growing group of blue characters auto-running down a long lane. The goal is to grow the crowd as large as possible, survive spinning hurdles and enemy-crowd battles, and reach the finish line with as many characters as possible.

Core loop

- Game starts: player character stands idle; the "Hold & Move" UI prompt is shown.
- The player holds and drags to steer the crowd left / right while it auto-runs forward.
- The crowd passes through paired gate triggers that add, subtract, multiply or divide the count.
- Rotating spike hurdles kill any individual character that touches them.
- Stationary enemy crowds block the path — a 1v1 battle removes one from each side per tick.
- The surviving crowd reaches the finish line and the end sequence plays.

Crowd formation

The crowd always occupies a filled disc, arranged in concentric rings. New characters are added to the outermost ring and evenly distributed around it; the radius grows with the count.

Ring 0 (centre)	1 character at local (0, 0)
Ring N radius	baseRadius + (N × ringSpacing)
Characters / ring	floor(2π × ringRadius / characterSpacing)
Placement	Evenly spaced around the ring using equal angle steps
New characters	Always fill the outermost ring first

All three values (baseRadius, ringSpacing, characterSpacing) are serialized floats on CrowdFormation.cs.

End sequence — Levels 1 & 2 (pyramid + stairs)

- Crowd hits the checkered finish line and stops moving.
- Characters stack into a bottom-heavy pyramid (e.g. 6 → rows 3, 2, 1).

- The pyramid advances toward the coloured multiplier stairs.
- Bottom-row characters step onto the lowest stair and stop; the next row steps onto the next stair, and so on.

End sequence — Level 3 (boss)

- No staircase. A large boss with a health bar waits at the end.
- The boss takes one hit per crowd character that reaches it; each character is consumed on impact.
- Win if boss HP reaches 0 before the crowd empties; lose otherwise.

3. Level Breakdown

Level	Contents	Ending
1	Rotating hurdles + enemy groups. Simpler layout, lower enemy counts.	Pyramid → stairs
2	Rotating hurdles + enemy groups. Harder layout, higher enemy counts.	Pyramid → stairs
3	Rotating hurdles + enemy groups + boss with health bar.	Boss fight (no stairs)

Platform elements (all levels)

- **Gate triggers** — pairs of gates, each with a hardcoded operator and value set in the Inspector. Operators: + Add, – Subtract, × Multiply, ÷ Divide.
- **Rotating hurdles** — purple spike cylinders that spin and kill any character on contact.
- **Enemy groups** — stationary red crowds that begin a 1v1 battle when the player crowd enters their trigger radius.

4. Folder Structure (Phase 1)

All game content lives under `Assets/_Game/` to keep it separate from third-party assets. Create every folder below before writing scripts.

```
Assets/
+-- _Game/
    +-- Animations/
    |   +-- Player/   (Idle.anim, Run.anim, Die.anim)
    |   +-- Enemy/    (Idle.anim, Die.anim)
    |   +-- Boss/     (Idle.anim, Hit.anim)
    +-- Materials/    (Player_Blue, Enemy_Red, Platform, Boss)
    +-- Prefabs/
    |   +-- Characters/  (PlayerCharacter, EnemyCharacter, BossCharacter)
    |   +-- Level/       (GateTrigger, EnemyGroup, SpinningHurdle,
    |   |                 FinishLine, Staircase)
    |   +-- UI/          (HoldMoveUI, CountBubble, BossHealthBar)
    +-- Scenes/       (MainMenu, Level1, Level2, Level3)
    +-- Scripts/
        +-- Managers/  (GameManager, LevelManager, UIManager)
        +-- Player/    (PlayerController, CrowdManager,
        |               CrowdFormation, PlayerCharacter)
        +-- Enemy/     (EnemyGroup, EnemyCharacter)
        +-- Boss/      (BossController)
        +-- Level/     (GateTrigger, SpinningHurdle,
        |               FinishLine, StaircaseSequence)
        +-- Pool/      (ObjectPool)
        +-- UI/        (HoldMoveUI, CountBubbleUI, BossHealthBarUI)
```

5. Project Setup (Phase 3 prerequisites)

Tags — create these tags and assign them as noted

- **Player** — assign to the *Crowd Root* object (the parent that moves and carries the trigger collider). Gates, hurdles, enemy groups, finish line and boss all check for this tag.

Layers & physics

- The Crowd Root needs a Rigidbody (Is Kinematic = true) and a trigger Collider so OnTriggerEnter fires on gates, the finish line, enemy groups and the boss arena.
- Individual PlayerCharacter prefabs need their own small trigger collider so SpinningHurdle can detect them.

Packages

- Import **TextMeshPro Essentials** (Window > TextMeshPro > Import TMP Essential Resources) — required by CountBubbleUI and the win/lose panels.

Camera

- One Main Camera tagged `MainCamera`, positioned behind and above the crowd root looking down the lane (third-person follow). CountBubbleUI billboards toward Camera.main.

6. Complete C# Scripts (Phase 2)

Create each script at the path shown in its header, using the exact contents below. These compile against a clean Unity 2022 project with TextMeshPro imported. Do not modify them.

6.1 Assets/_Game/Scripts/Managers/GameManager.cs

```
using UnityEngine;

// Singleton that owns the overall game state for a single level.
// States: Idle (waiting for first touch), Playing, Win, Lose.
public class GameManager : MonoBehaviour
{
    // The one and only instance of this manager.
    public static GameManager Instance;

    public enum GameState { Idle, Playing, Win, Lose }

    // Current state, visible in the Inspector for debugging.
    [SerializeField] private GameState state = GameState.Idle;

    // Sets up the singleton reference. Runs before Start.
    private void Awake()
    {
        // If another instance already exists, destroy this one.
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
    }

    // Returns the current game state.
    public GameState GetState()
    {
        return state;
    }

    // Called by PlayerController on the very first touch to begin play.
    public void StartPlaying()
    {
        // Only move from Idle into Playing once.
        if (state == GameState.Idle)
        {
            state = GameState.Playing;
            UIManager.Instance.HideHoldMove();
        }
    }

    // Called when the player wins (finished stairs or killed the boss).
    public void Win()
    {
        // Guard so we only trigger the win flow once.
        if (state == GameState.Win || state == GameState.Lose) return;
        state = GameState.Win;
        UIManager.Instance.ShowWinPanel();
    }

    // Called when the player loses (crowd wiped out).
    public void Lose()
    {
        // Guard so we only trigger the lose flow once.
        if (state == GameState.Win || state == GameState.Lose) return;
        state = GameState.Lose;
        UIManager.Instance.ShowLosePanel();
    }
}
```

6.2 Assets/_Game/Scripts/Managers/LevelManager.cs

```
using UnityEngine;
using UnityEngine.SceneManagement;

// Singleton that handles loading, reloading and advancing between levels.
public class LevelManager : MonoBehaviour
{
    // The one and only instance of this manager.
    public static LevelManager Instance;
    // Scene names in build order. Set these in the Inspector.
```

```
    [SerializeField] private string[] levelSceneNames =
        { "Level1", "Level2", "Level3" };

    // Index of the level currently loaded (0-based).
    [SerializeField] private int currentLevelIndex = 0;

    // Sets up the singleton and keeps it alive across scene loads.
    private void Awake()
    {
        // Enforce a single persistent instance.
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
        DontDestroyOnLoad(gameObject);
    }

    // Loads the next level, or loops back to the first after the last one.
    public void LoadNextLevel()
    {
        currentLevelIndex++;
        // Wrap around if we passed the final level.
        if (currentLevelIndex >= levelSceneNames.Length)
        {
            currentLevelIndex = 0;
        }
        SceneManager.LoadScene(levelSceneNames[currentLevelIndex]);
    }

    // Reloads the current level (used after a loss).
    public void ReloadLevel()
    {
        SceneManager.LoadScene(levelSceneNames[currentLevelIndex]);
    }

    // Loads a specific level by its index (used by a menu, if any).
    public void LoadLevel(int index)
    {
        // Ignore out-of-range requests.
        if (index < 0 || index >= levelSceneNames.Length) return;
        currentLevelIndex = index;
        SceneManager.LoadScene(levelSceneNames[index]);
    }
}
```

6.3 Assets/_Game/Scripts/Managers/UIManager.cs

```
using UnityEngine;

// Singleton that shows and hides all the UI panels and widgets.
public class UIManager : MonoBehaviour
{
    // The one and only instance of this manager.
    public static UIManager Instance;

    [Header("UI References (assign in Inspector)")]
    // The "Hold & Move" prompt shown at the start.
    [SerializeField] private GameObject holdMovePanel;
    // Panel shown when the player wins.
    [SerializeField] private GameObject winPanel;
    // Panel shown when the player loses.
    [SerializeField] private GameObject losePanel;
    // Root object of the boss health bar (level 3 only).
    [SerializeField] private GameObject bossHealthBarRoot;

    // Sets up the singleton reference.
    private void Awake()
    {
        // Enforce a single instance.
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
    }

    // Ensures panels start in the right visibility at scene load.
    private void Start()
    {
        // Hold & Move visible, everything else hidden.
        if (holdMovePanel != null) holdMovePanel.SetActive(true);
```

```
        if (winPanel != null) winPanel.SetActive(false);
        if (losePanel != null) losePanel.SetActive(false);
        if (bossHealthBarRoot != null) bossHealthBarRoot.SetActive(false);
    }

    // Hides the "Hold & Move" prompt (called on first touch).
    public void HideHoldMove()
    {
        if (holdMovePanel != null) holdMovePanel.SetActive(false);
    }

    // Shows the win panel.
    public void ShowWinPanel()
    {
        if (winPanel != null) winPanel.SetActive(true);
    }

    // Shows the lose panel.
    public void ShowLosePanel()
    {
        if (losePanel != null) losePanel.SetActive(true);
    }

    // Reveals the boss health bar (level 3 arena entry).
    public void ShowBossHealthBar()
    {
        if (bossHealthBarRoot != null) bossHealthBarRoot.SetActive(true);
    }
}
```

6.4 Assets/_Game/Scripts/Pool/ObjectPool.cs

```
using System.Collections.Generic;
using UnityEngine;

// Simple object pool for PlayerCharacter instances.
// Pre-warms a set of characters at scene start so we never
// call Instantiate/Destroy during gameplay (better for mobile).
public class ObjectPool : MonoBehaviour
{
    // The one and only instance of this pool.
    public static ObjectPool Instance;

    [Header("Pool Settings")]
    // The PlayerCharacter prefab to pool.
    [SerializeField] private PlayerCharacter characterPrefab;
    // How many characters to create up front.
    [SerializeField] private int prewarmCount = 300;

    // The list of currently inactive (available) characters.
    private readonly Queue<PlayerCharacter> available =
        new Queue<PlayerCharacter>();

    // Sets up the singleton reference.
    private void Awake()
    {
        // Enforce a single instance.
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
    }

    // Creates the initial batch of pooled characters.
    private void Start()
    {
        // Build the requested number of inactive characters.
        for (int i = 0; i < prewarmCount; i++)
        {
            PlayerCharacter pc = Instantiate(characterPrefab, transform);
            pc.gameObject.SetActive(false);
            available.Enqueue(pc);
        }
    }

    // Returns an available character, expanding the pool if empty.
    public PlayerCharacter Get()
    {
        PlayerCharacter pc;
        // Grow the pool on demand if we ran dry.
        if (available.Count > 0)
        {
```

```
            pc = available.Dequeue();
        }
        else
        {
            pc = Instantiate(characterPrefab, transform);
        }
        pc.gameObject.SetActive(true);
        return pc;
    }

    // Returns a character back to the pool for reuse.
    public void Return(PlayerCharacter pc)
    {
        // Deactivate and store it for the next Get().
        pc.gameObject.SetActive(false);
        pc.transform.SetParent(transform);
        available.Enqueue(pc);
    }
}
```

6.5 Assets/_Game/Scripts/Player/PlayerCharacter.cs

```
using System.Collections;
using UnityEngine;

// Represents a single blue character inside the player crowd.
// Handles its own death animation and return to the pool.
public class PlayerCharacter : MonoBehaviour
{
    [Header("References")]
    // Animator that drives Idle / Run / Die states.
    [SerializeField] private Animator animator;

    // How long the die animation plays before returning to the pool.
    [SerializeField] private float dieDelay = 0.4f;

    // Cached name of the running bool parameter on the Animator.
    private const string RunParam = "IsRunning";
    private const string DieTrigger = "Die";

    // Tells the character to start or stop the run animation.
    public void SetRunning(bool isRunning)
    {
        // Guard against a missing animator reference.
        if (animator != null) animator.SetBool(RunParam, isRunning);
    }

    // Kills this character: plays die anim then returns to the pool.
    public void Die()
    {
        // Remove from the crowd list immediately so counts stay correct.
        CrowdManager.Instance.UnregisterCharacter(this);
        StartCoroutine(DieRoutine());
    }

    // Waits for the die animation, then recycles this object.
    private IEnumerator DieRoutine()
    {
        // Trigger the death animation if available.
        if (animator != null) animator.SetTrigger(DieTrigger);
        yield return new WaitForSeconds(dieDelay);
        // Reset running state for the next time it is reused.
        SetRunning(false);
        ObjectPool.Instance.Return(this);
    }
}
```

6.6 Assets/_Game/Scripts/Player/CrowdFormation.cs

```
using System.Collections.Generic;
using UnityEngine;

// Computes concentric-ring (filled disc) local positions for N characters
// and smoothly moves each character toward its assigned slot.
public class CrowdFormation : MonoBehaviour
{
    [Header("Formation Tuning")]
    // Radius of the first ring out from the centre.
    [SerializeField] private float baseRadius = 0.4f;
    // Extra radius added for each ring further out.
    [SerializeField] private float ringSpacing = 0.5f;
    // Approximate spacing between characters along a ring.
    [SerializeField] private float characterSpacing = 0.55f;
    // How quickly characters slide to their target slots.
```

```
    [SerializeField] private float moveLerpSpeed = 10f;

    // Target local position for each character index.
    private readonly List<Vector3> slotPositions = new List<Vector3>();

    // Recomputes all slot positions for the given character count.
    public void RecomputeSlots(int count)
    {
        // Clear any previous layout.
        slotPositions.Clear();
        if (count <= 0) return;

        // The very first character sits at the centre.
        slotPositions.Add(Vector3.zero);
        int placed = 1;
        int ring = 1;

        // Keep adding rings until every character has a slot.
        while (placed < count)
        {
            float ringRadius = baseRadius + (ring * ringSpacing);
            // Number of characters that fit on this ring's circumference.
            int perRing = Mathf.Max(1,
                Mathf.FloorToInt((2f * Mathf.PI * ringRadius) / characterSpacing));

            // Place characters evenly around this ring.
            for (int i = 0; i < perRing && placed < count; i++)
            {
                float angle = (2f * Mathf.PI / perRing) * i;
                float x = Mathf.Cos(angle) * ringRadius;
                float z = Mathf.Sin(angle) * ringRadius;
                slotPositions.Add(new Vector3(x, 0f, z));
                placed++;
            }
            ring++;
        }
    }

    // Smoothly moves each character toward its slot every frame.
    // Called by CrowdManager in its Update.
    public void UpdatePositions(List<PlayerCharacter> characters)
    {
        // Move as many characters as we have slots for.
        for (int i = 0; i < characters.Count && i < slotPositions.Count; i++)
        {
            Transform t = characters[i].transform;
            // Lerp local position so the disc keeps its shape as it moves.
            t.localPosition = Vector3.Lerp(
                t.localPosition,
                slotPositions[i],
                Time.deltaTime * moveLerpSpeed);
        }
    }
}
```

6.7 Assets/_Game/Scripts/Player/CrowdManager.cs

```
using System.Collections.Generic;
using UnityEngine;

// Singleton that owns the list of active player characters and performs
// all count operations (add / subtract / multiply / divide).
public class CrowdManager : MonoBehaviour
{
    // The one and only instance of this manager.
    public static CrowdManager Instance;

    [Header("References")]
    // Formation helper that arranges characters into a disc.
    [SerializeField] private CrowdFormation formation;
    // World-space count bubble UI following the crowd.
    [SerializeField] private CountBubbleUI countBubble;

    [Header("Start Settings")]
    // How many characters the crowd begins with.
    [SerializeField] private int startCount = 1;
    // Safety cap so the pool and formation never explode.
    [SerializeField] private int maxCount = 300;

    // The list of currently active characters.
    private readonly List<PlayerCharacter> characters =
        new List<PlayerCharacter>();

    // Sets up the singleton reference.
```

```
private void Awake()
{
    // Enforce a single instance.
    if (Instance != null && Instance != this)
    {
        Destroy(gameObject);
        return;
    }
    Instance = this;
}

// Spawns the starting characters once the pool is ready.
private void Start()
{
    // Add the initial crowd members.
    AddCharacters(startCount);
}

// Keeps the disc formation updated every frame.
private void Update()
{
    formation.UpdatePositions(characters);
}

// Returns the current number of active characters.
public int GetCount()
{
    return characters.Count;
}

// Adds a number of characters from the pool to the crowd.
public void AddCharacters(int amount)
{
    // Add one at a time, respecting the max cap.
    for (int i = 0; i < amount; i++)
    {
        if (characters.Count >= maxCount) break;
        PlayerCharacter pc = ObjectPool.Instance.Get();
        // Parent under the crowd root so it moves with the group.
        pc.transform.SetParent(transform, false);
        pc.transform.localPosition = Vector3.zero;
        pc.SetRunning(GameManager.Instance.GetState()
            == GameManager.GameState.Playing);
        characters.Add(pc);
    }
    RefreshAfterChange();
}

// Removes a number of characters by killing them.
public void RemoveCharacters(int amount)
{
    // Kill from the end of the list.
    for (int i = 0; i < amount; i++)
    {
        if (characters.Count == 0) break;
        PlayerCharacter pc = characters[characters.Count - 1];
        // Die() calls UnregisterCharacter which updates the list.
        pc.Die();
    }
    // Note: Die() already refreshes via UnregisterCharacter.
}

// Multiplies the crowd size by a factor (used by x gates).
public void MultiplyCrowd(float factor)
{
    int current = characters.Count;
    int target = Mathf.FloorToInt(current * factor);
    int toAdd = target - current;
    // Only positive growth is expected from multiply gates.
    if (toAdd > 0) AddCharacters(toAdd);
}

// Divides the crowd size by a divisor (used by divide gates).
public void DivideCrowd(float divisor)
{
    // Ignore invalid divisors.
    if (divisor <= 0f) return;
    int current = characters.Count;
    int target = Mathf.FloorToInt(current / divisor);
    int toRemove = current - target;
    if (toRemove > 0) RemoveCharacters(toRemove);
}
// Called by PlayerCharacter.Die() to remove itself from the list.
```

```
    public void UnregisterCharacter(PlayerCharacter pc)
    {
        // Remove and refresh the layout / count bubble.
        if (characters.Remove(pc))
        {
            RefreshAfterChange();
            // Losing the whole crowd is a loss.
            if (characters.Count == 0)
            {
                GameManager.Instance.Lose();
            }
        }
    }

    // Returns a copy of the current character list (for stairs / boss).
    public List<PlayerCharacter> GetCharacters()
    {
        return new List<PlayerCharacter>(characters);
    }

    // Recomputes the disc slots and updates the count bubble.
    private void RefreshAfterChange()
    {
        formation.RecomputeSlots(characters.Count);
        if (countBubble != null) countBubble.SetCount(characters.Count);
    }
}
```

6.8 Assets/_Game/Scripts/Player/PlayerController.cs

```
using UnityEngine;

// Reads touch / mouse drag input, auto-runs the crowd root forward,
// and steers it left / right within the lane.
public class PlayerController : MonoBehaviour
{
    [Header("Movement Tuning")]
    // Forward speed once the game is playing.
    [SerializeField] private float forwardSpeed = 6f;
    // How fast dragging moves the crowd sideways.
    [SerializeField] private float steerSpeed = 12f;
    // Half the lane width; clamps left/right position.
    [SerializeField] private float laneHalfWidth = 3f;

    // Screen X of the finger/mouse on the previous frame.
    private float lastPointerX;
    // Whether we currently have a finger/mouse held down.
    private bool isDragging;

    // Reads input each frame and moves the crowd root.
    private void Update()
    {
        // Only respond while idle (to start) or actively playing.
        GameManager.GameState state = GameManager.Instance.GetState();
        if (state == GameManager.GameState.Win
            || state == GameManager.GameState.Lose) return;

        HandlePointer(state);

        // Auto-run forward only after play has started.
        if (state == GameManager.GameState.Playing)
        {
            transform.Translate(Vector3.forward * forwardSpeed * Time.deltaTime,
                Space.World);
        }
    }

    // Handles press / drag / release for both touch and mouse.
    private void HandlePointer(GameManager.GameState state)
    {
        // Press began this frame.
        if (Input.GetMouseButtonDown(0))
        {
            isDragging = true;
            lastPointerX = Input.mousePosition.x;
            // First touch starts the game.
            if (state == GameManager.GameState.Idle)
            {
                GameManager.Instance.StartPlaying();
                SetCrowdRunning(true);
            }
        }
        // Press released this frame.
        else if (Input.GetMouseButtonUp(0))
```

```
        {
            isDragging = false;
        }

        // While held, steer based on horizontal finger movement.
        if (isDragging)
        {
            float deltaX = Input.mousePosition.x - lastPointerX;
            lastPointerX = Input.mousePosition.x;

            // Convert screen delta into world sideways movement.
            float move = (deltaX / Screen.width) * steerSpeed;
            Vector3 pos = transform.position;
            pos.x = Mathf.Clamp(pos.x + move, -laneHalfWidth, laneHalfWidth);
            transform.position = pos;
        }
    }

    // Tells every crowd character to play the run animation.
    private void SetCrowdRunning(bool running)
    {
        // Ask each active character to switch animation state.
        foreach (PlayerCharacter pc in CrowdManager.Instance.GetCharacters())
        {
            pc.SetRunning(running);
        }
    }
}
```

6.9 Assets/_Game/Scripts/Level/GateTrigger.cs

```
using UnityEngine;

// A single gate the crowd runs through. The operator and value are
// hardcoded per-prefab-instance in the Inspector (no randomisation).
public class GateTrigger : MonoBehaviour
{
    public enum Operator { Add, Subtract, Multiply, Divide }

    [Header("Gate Settings (set per gate in Inspector)")]
    // Which maths operation this gate applies.
    [SerializeField] private Operator op = Operator.Add;
    // The value used by the operation (e.g. 25 for +25, 3 for x3).
    [SerializeField] private float value = 25f;

    // Ensures the gate only fires once per crowd pass.
    private bool consumed;

    // Fires when the crowd root (tagged "Player") enters the gate.
    private void OnTriggerEnter(Collider other)
    {
        // Ignore anything that is not the crowd root, and fire only once.
        if (consumed) return;
        if (!other.CompareTag("Player")) return;

        consumed = true;
        ApplyOperation();
    }

    // Calls the matching CrowdManager method based on the operator.
    private void ApplyOperation()
    {
        // Route to the correct crowd operation.
        switch (op)
        {
            case Operator.Add:
                CrowdManager.Instance.AddCharacters(Mathf.RoundToInt(value));
                break;
            case Operator.Subtract:
                CrowdManager.Instance.RemoveCharacters(Mathf.RoundToInt(value));
                break;
            case Operator.Multiply:
                CrowdManager.Instance.MultiplyCrowd(value);
                break;
            case Operator.Divide:
                CrowdManager.Instance.DivideCrowd(value);
                break;
        }
    }
}
```

6.10 Assets/_Game/Scripts/Level/SpinningHurdle.cs

```
using UnityEngine;

// A rotating purple spike cylinder. Any player character that touches it dies.
public class SpinningHurdle : MonoBehaviour
{
    [Header("Rotation")]
    // Degrees per second around the Y axis.
    [SerializeField] private float rotateSpeed = 120f;

    // Spins the hurdle every frame.
    private void Update()
    {
        transform.Rotate(Vector3.up, rotateSpeed * Time.deltaTime, Space.World);
    }

    // Kills any individual player character that collides with the spikes.
    private void OnTriggerEnter(Collider other)
    {
        // Only react to individual crowd characters.
        PlayerCharacter pc = other.GetComponent<PlayerCharacter>();
        if (pc != null)
        {
            pc.Die();
        }
    }
}
```

6.11 Assets/_Game/Scripts/Enemy/EnemyCharacter.cs

```
using System.Collections;
using UnityEngine;

// Represents a single red enemy character in an enemy group.
public class EnemyCharacter : MonoBehaviour
{
    [Header("References")]
    // Animator driving Idle / Die states.
    [SerializeField] private Animator animator;
    // How long the die animation plays before the object is destroyed.
    [SerializeField] private float dieDelay = 0.4f;

    private const string DieTrigger = "Die";

    // Plays the death animation then removes this enemy from the scene.
    public void Die()
    {
        StartCoroutine(DieRoutine());
    }

    // Waits for the die animation, then destroys the enemy object.
    private IEnumerator DieRoutine()
    {
        // Trigger death animation if present.
        if (animator != null) animator.SetTrigger(DieTrigger);
        yield return new WaitForSeconds(dieDelay);
        Destroy(gameObject);
    }
}
```

6.12 Assets/_Game/Scripts/Enemy/EnemyGroup.cs

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// A stationary red crowd. When the player crowd enters the trigger radius,
// a battle begins: one player and one enemy die per tick until one side
// is wiped out.
public class EnemyGroup : MonoBehaviour
{
    [Header("Battle Settings")]
    // Seconds between each 1v1 exchange.
    [SerializeField] private float hitInterval = 0.05f;

    // All enemy characters that belong to this group.
    private readonly List<EnemyCharacter> enemies =
        new List<EnemyCharacter>();

    // Ensures the battle only starts once.
    private bool battleStarted;

    // Collects all child EnemyCharacter components at start.
```

```
    private void Start()
    {
        // Gather every enemy nested under this group.
        GetComponentsInChildren(true, enemies);
    }

    // Starts the battle when the player crowd enters the radius.
    private void OnTriggerEnter(Collider other)
    {
        // Only the crowd root (tagged "Player") should start the battle.
        if (battleStarted) return;
        if (!other.CompareTag("Player")) return;

        battleStarted = true;
        StartCoroutine(BattleRoutine());
    }

    // Removes one enemy and one player per tick until a side is empty.
    private IEnumerator BattleRoutine()
    {
        // Keep fighting while both sides still have members.
        while (enemies.Count > 0 && CrowdManager.Instance.GetCount() > 0)
        {
            // Kill one enemy from this group.
            EnemyCharacter enemy = enemies[enemies.Count - 1];
            enemies.RemoveAt(enemies.Count - 1);
            if (enemy != null) enemy.Die();

            // Kill one player from the crowd.
            CrowdManager.Instance.RemoveCharacters(1);

            yield return new WaitForSeconds(hitInterval);
        }
        // If the crowd is empty, CrowdManager already calls Lose().
    }
}
```

6.13 Assets/_Game/Scripts/Level/FinishLine.cs

```
using UnityEngine;

// Detects the crowd crossing the checkered line. On levels 1 & 2 it starts
// the staircase sequence; on level 3 it activates the boss instead.
public class FinishLine : MonoBehaviour
{
    [Header("End-of-level References")]
    // Staircase sequence for levels 1 & 2 (leave empty on level 3).
    [SerializeField] private StaircaseSequence staircase;
    // Boss controller for level 3 (leave empty on levels 1 & 2).
    [SerializeField] private BossController boss;

    // Ensures the finish logic runs only once.
    private bool finished;

    // Fires when the crowd root crosses the finish line.
    private void OnTriggerEnter(Collider other)
    {
        // Only the crowd root triggers the finish, and only once.
        if (finished) return;
        if (!other.CompareTag("Player")) return;

        finished = true;
        BeginEndSequence();
    }

    // Chooses between the staircase (1 & 2) or the boss (3).
    private void BeginEndSequence()
    {
        // If a staircase is assigned, run the pyramid + stairs flow.
        if (staircase != null)
        {
            staircase.BeginSequence();
        }
        // Otherwise, if a boss is assigned, start the boss fight.
        else if (boss != null)
        {
            boss.BeginFight();
        }
    }
}
```

6.14 Assets/_Game/Scripts/Level/StaircaseSequence.cs

```
using System.Collections;
```

```
using System.Collections.Generic;
using UnityEngine;

// Levels 1 & 2 only. Arranges the surviving crowd into a bottom-heavy
// pyramid, then walks each row up onto a coloured multiplier stair.
public class StaircaseSequence : MonoBehaviour
{
    [Header("Stair References")]
    // World positions of each stair step, lowest first.
    [SerializeField] private Transform[] stairPoints;
    // How fast characters walk to their stair positions.
    [SerializeField] private float walkSpeed = 4f;
    // Time between placing each row.
    [SerializeField] private float rowDelay = 0.4f;

    // Called by FinishLine when the crowd crosses the line.
    public void BeginSequence()
    {
        StartCoroutine(SequenceRoutine());
    }

    // Builds pyramid rows and walks each row onto its stair.
    private IEnumerator SequenceRoutine()
    {
        // Snapshot the surviving characters.
        List<PlayerCharacter> crowd = CrowdManager.Instance.GetCharacters();
        List<int> rows = BuildPyramidRows(crowd.Count);

        int index = 0;
        // Place each row on the next stair, lowest first.
        for (int r = 0; r < rows.Count; r++)
        {
            // Stop if we run out of physical stairs.
            if (r >= stairPoints.Length) break;

            int rowSize = rows[r];
            Transform stair = stairPoints[r];

            // Spread this row's characters across the stair width.
            for (int c = 0; c < rowSize && index < crowd.Count; c++)
            {
                float offset = (c - (rowSize - 1) / 2f) * 0.5f;
                Vector3 target = stair.position + stair.right * offset;
                StartCoroutine(WalkTo(crowd[index].transform, target));
                index++;
            }
            yield return new WaitForSeconds(rowDelay);
        }

        // Small pause, then declare victory.
        yield return new WaitForSeconds(1f);
        GameManager.Instance.Win();
    }

    // Computes pyramid row sizes: bottom row biggest, minus 1 each row up.
    private List<int> BuildPyramidRows(int count)
    {
        // Example: 6 -> 3,2,1 ; 4 -> 3,1 ; 3 -> 2,1.
        List<int> rows = new List<int>();
        int rowSize = Mathf.Max(1, Mathf.CeilToInt(Mathf.Sqrt(count)));
        int remaining = count;
        // Fill each row, reducing by one, until characters run out.
        while (remaining > 0 && rowSize > 0)
        {
            int take = Mathf.Min(rowSize, remaining);
            rows.Add(take);
            remaining -= take;
            rowSize--;
        }
        return rows;
    }

    // Smoothly walks a single character to a target world position.
    private IEnumerator WalkTo(Transform character, Vector3 target)
    {
        // Detach from the crowd root so it can move independently.
        character.SetParent(null);
        // Move until close enough to the target.
        while (Vector3.Distance(character.position, target) > 0.05f)
        {
            character.position = Vector3.MoveTowards(
                character.position, target, walkSpeed * Time.deltaTime);
            yield return null;
```

```
        }
    }
}
```

6.15 Assets/_Game/Scripts/Boss/BossController.cs

```
using System.Collections;
using UnityEngine;

// Level 3 only. A large enemy with a health bar. Each crowd character that
// reaches it deals one damage and is consumed. Win when boss HP hits 0;
// lose if the crowd empties first.
public class BossController : MonoBehaviour
{
    [Header("Boss Settings")]
    // Total boss health (number of hits it can take).
    [SerializeField] private int maxHealth = 70;
    // Seconds between each crowd character striking the boss.
    [SerializeField] private float hitInterval = 0.05f;
    // Health bar UI to update each hit.
    [SerializeField] private BossHealthBarUI healthBar;

    // Current boss health.
    private int currentHealth;
    // Ensures the fight only begins once.
    private bool fightStarted;

    // Called by FinishLine when the crowd enters the arena.
    public void BeginFight()
    {
        // Guard against multiple starts.
        if (fightStarted) return;
        fightStarted = true;

        currentHealth = maxHealth;
        UIManager.Instance.ShowBossHealthBar();
        if (healthBar != null) healthBar.SetFill(1f);

        StartCoroutine(FightRoutine());
    }

    // Consumes one crowd member per tick, damaging the boss each time.
    private IEnumerator FightRoutine()
    {
        // Fight until the boss dies or the crowd is wiped out.
        while (currentHealth > 0 && CrowdManager.Instance.GetCount() > 0)
        {
            // One character strikes and is consumed.
            CrowdManager.Instance.RemoveCharacters(1);
            currentHealth--;

            // Update the health bar fill amount.
            if (healthBar != null)
            {
                healthBar.SetFill((float)currentHealth / maxHealth);
            }

            yield return new WaitForSeconds(hitInterval);
        }

        // Decide the outcome.
        if (currentHealth <= 0)
        {
            GameManager.Instance.Win();
        }
        else
        {
            GameManager.Instance.Lose();
        }
    }
}
```

6.16 Assets/_Game/Scripts/UI/HoldMoveUI.cs

```
using UnityEngine;

// Shows the "Hold & Move" prompt and hides itself on first touch.
// UIManager also hides it via HideHoldMove(); this script is a simple
// self-contained fallback you can attach directly to the prompt object.
public class HoldMoveUI : MonoBehaviour
{
    // Hides the prompt the moment the game leaves the Idle state.
    private void Update()
    {
        // Once play has begun, hide this prompt and stop updating.
        if (GameManager.Instance.GetState() != GameManager.GameState.Idle)
        {
            gameObject.SetActive(false);
        }
    }
}
```

6.17 Assets/_Game/Scripts/UI/CountBubbleUI.cs

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;

// A world-space bubble that follows the crowd root and shows the count.
public class CountBubbleUI : MonoBehaviour
{
    [Header("References")]
    // The crowd root to follow.
    [SerializeField] private Transform crowdRoot;
    // Text component showing the number (TextMeshPro).
    [SerializeField] private TMP_Text countText;
    // Height above the crowd root to float the bubble.
    [SerializeField] private float heightOffset = 2.2f;

    // Follows the crowd root each frame and faces the camera.
    private void LateUpdate()
    {
        // Keep the bubble above the crowd root.
        if (crowdRoot != null)
        {
            transform.position = crowdRoot.position + Vector3.up * heightOffset;
        }
        // Billboard the bubble toward the camera.
        if (Camera.main != null)
        {
            transform.forward = Camera.main.transform.forward;
        }
    }

    // Updates the displayed count. Called by CrowdManager.
    public void SetCount(int count)
    {
        if (countText != null) countText.text = count.ToString();
    }
}
```

6.18 Assets/_Game/Scripts/UI/BossHealthBarUI.cs

```
using UnityEngine;
using UnityEngine.UI;

// Level 3 only. Shows boss HP as a horizontal fill bar.
public class BossHealthBarUI : MonoBehaviour
{
    [Header("References")]
    // The Image set to Filled / Horizontal used as the red bar.
    [SerializeField] private Image fillImage;

    // Sets the fill amount (0 = empty, 1 = full). Called by BossController.
    public void SetFill(float normalized)
    {
        // Clamp and apply the fill.
        if (fillImage != null)
        {
            fillImage.fillAmount = Mathf.Clamp01(normalized);
        }
    }
}
```

7. Prefab Assembly (Phase 4)

Placeholder art is fine (primitives). Build each prefab, attach the listed component(s), then save into the matching Prefabs subfolder.

7.1 PlayerCharacter (Prefabs/Characters)

- Capsule mesh, Player_Blue material.
- Small trigger **Collider** (Is Trigger = true) so hurdles can detect it.
- **Animator** with Idle / Run / Die states, a bool `IsRunning` and a trigger `Die`.
- Attach **PlayerCharacter.cs**; assign its Animator field.

7.2 EnemyCharacter (Prefabs/Characters)

- Capsule mesh, Enemy_Red material.
- **Animator** with Idle / Die states and a trigger `Die`.
- Attach **EnemyCharacter.cs**; assign its Animator field.

7.3 BossCharacter (Prefabs/Characters)

- Larger character mesh, Boss material; place at the level-3 arena end.
- Attach **BossController.cs**; set Max Health (e.g. 70) and Hit Interval; assign the BossHealthBar reference (7.9).

7.4 GateTrigger (Prefabs/Level)

- A flat gate quad/cube with a **trigger box collider** spanning one half of the lane.
- A world-space TextMeshPro label showing the operator + value (e.g. "+25", "x3").
- Attach **GateTrigger.cs**; set Operator and Value per instance in the Inspector. Gates are placed in pairs (left + right).

7.5 SpinningHurdle (Prefabs/Level)

- Purple cylinder with spike detail; a **trigger collider** around it.
- Attach **SpinningHurdle.cs**; set Rotate Speed (e.g. 120).

7.6 EnemyGroup (Prefabs/Level)

- An empty root with a **trigger collider** (the battle radius) and N EnemyCharacter children arranged in a disc.
- Attach **EnemyGroup.cs**; set Hit Interval. It auto-collects its EnemyCharacter children at Start.
- A world-space count label (red bubble) above the group is optional but matches the screenshots.

7.7 FinishLine (Prefabs/Level)

- Checkered strip across the lane with a **trigger collider**.
- Attach **FinishLine.cs**. On levels 1 & 2 assign the Staircase field; on level 3 assign the Boss field (leave the other empty).

7.8 Staircase (Prefabs/Level)

- A run of coloured steps, each with an empty child **Transform** marking where a pyramid row lands.
- Attach **StaircaseSequence.cs**; drag the step Transforms into the Stair Points array (lowest first).

7.9 UI prefabs (Prefabs/UI)

- **HoldMoveUI** — a Canvas element with the "Hold & Move" prompt; attach HoldMoveUI.cs (optional self-hide fallback).
- **CountBubble** — world-space bubble with a TMP text; attach CountBubbleUI.cs; assign Crowd Root and the TMP text.
- **BossHealthBar** — a UI Image set to *Filled* / *Horizontal*; attach BossHealthBarUI.cs; assign the Fill Image.

8. Scene Assembly (Phase 5)

Common setup for every level scene

- Create empty GameObjects for the managers and attach: **GameManager**, **UIManager**, **ObjectPool**. (LevelManager lives in MainMenu with DontDestroyOnLoad, or place one per scene for testing.)
- Create a **Crowd Root** empty object at the lane start: tag `Player`, add a kinematic Rigidbody + trigger collider, and attach **PlayerController**, **CrowdManager** and **CrowdFormation**.
- Add the Main Camera as a follow camera behind the Crowd Root.
- Add a Canvas with the HoldMoveUI prompt, win panel, lose panel; add the world-space CountBubble.
- Lay a long lane (platform) mesh with side walls.

Level 1 — lane layout (front → back)

- Start zone (Crowd Root, idle).
- Gate pair A: e.g. left **+125** / right **+25**.
- 1–2 SpinningHurdles.
- One small EnemyGroup (low count, e.g. 30–50).
- Gate pair B (a mix, e.g. **+70** / **+80**).
- FinishLine (Staircase assigned) → Staircase with multiplier steps.

Level 2 — same recipe, harder

- More gate pairs including a Multiply gate (e.g. **x4** / **x5**) and a Divide gate for risk.
- More hurdles and 2–3 larger EnemyGroups (higher counts, e.g. 80+).
- FinishLine (Staircase assigned) → Staircase.

Level 3 — boss ending

- Hurdles + gates + enemy groups as before (can include a mixed gate like **x3** / **+60**).
- FinishLine with the **Boss** field assigned (Staircase left empty).
- BossCharacter in a circular arena past the finish line; BossHealthBar hidden until the fight starts.

9. Inspector Wiring Reference (Phase 6)

Every serialized reference the agent must connect. Missing links are the most common failure — verify each row.

Component	Field	Assign to
UIManager	Hold Move Panel / Win / Lose / Boss Bar Root	The matching Canvas objects
LevelManager	Level Scene Names	{ Level1, Level2, Level3 } (add to Build Settings)
ObjectPool	Character Prefab	PlayerCharacter prefab
ObjectPool	Prewarm Count	e.g. 300 (>= expected max crowd)
CrowdManager	Formation	The CrowdFormation on the Crowd Root
CrowdManager	Count Bubble	The CountBubble UI

Component	Field	Assign to
CrowdManager	Start Count / Max Count	1 / 300
PlayerCharacter	Animator	Its own Animator
EnemyCharacter	Animator	Its own Animator
GateTrigger	Operator / Value	Set per gate (e.g. Add / 25)
SpinningHurdle	Rotate Speed	e.g. 120
EnemyGroup	Hit Interval	e.g. 0.05
FinishLine	Staircase (L1/L2) OR Boss (L3)	Assign one, leave the other empty
StaircaseSequence	Stair Points	Step marker Transforms, lowest first
BossController	Max Health / Health Bar	e.g. 70 / the BossHealthBar UI
CountBubbleUI	Crowd Root / Count Text	Crowd Root / its TMP text
BossHealthBarUI	Fill Image	The Filled-Horizontal Image

10. Acceptance Checklist (Phase 6)

The build is complete when every item passes in Play mode.

- ■ **1.** Project opens in Unity 2022 with Built-in RP; no console errors; platform set to Android.
- ■ **2.** On play, one idle character stands at the lane start and the "Hold & Move" prompt is visible.
- ■ **3.** First touch hides the prompt, starts the run animation, and the crowd auto-runs forward.
- ■ **4.** Holding and dragging steers the crowd left/right, clamped within the lane.
- ■ **5.** Passing an Add gate increases the count; Subtract decreases; Multiply grows; Divide shrinks.
- ■ **6.** The crowd always re-packs into a filled disc after every count change.
- ■ **7.** The world-space count bubble always shows the correct current count.
- ■ **8.** Touching a spinning hurdle removes exactly the characters that contacted it (they play Die and return to the pool).
- ■ **9.** Entering an enemy group starts a 1v1 battle that removes one from each side per tick until a side is empty.
- ■ **10.** If the crowd hits zero at any point, the lose panel shows.
- ■ **11.** Levels 1 & 2: crossing the finish line stacks a pyramid and walks rows onto the stairs, then shows win.
- ■ **12.** Level 3: crossing the finish line reveals the boss health bar; characters are consumed one per hit; win at 0 HP, lose if crowd empties first.
- ■ **13.** PlayerCharacters are reused from the pool (no Instantiate/Destroy spam mid-run).
- ■ **14.** All managers expose a single static Instance and there are no duplicates in the scene.